Django Readiness Challenge Set — Code Solutions

These are simplified educational implementations designed to demonstrate the programming logic, OOP structure, validation, SQL thinking, and backend reasoning expected before starting Django development.

1. School Management System

```
class Student:
    def __init__(self, student_id, fullname, age, email, department):
        self.student_id = student_id
        self.fullname = fullname
        self.age = age
        self.email = email
        self.department = department
        self.courses = []

    def enroll_course(self, course):
        self.courses.append(course)

    def display(self):
        print(f"{self.fullname} - {self.department}")


class Teacher:
    def __init__(self, name):
        self.name = name
        self.courses = []

    def assign_course(self, course):
        self.courses.append(course)


class Course:
    def __init__(self, code, title, unit):
        self.code = code
        self.title = title
        self.unit = unit


students = []

def register_student():
    student_id = input("Enter ID: ")
    fullname = input("Enter fullname: ")
    age = int(input("Enter age: "))
    email = input("Enter email: ")
    department = input("Enter department: ")

    for student in students:
        if student.email == email:
            print("Duplicate email detected")
            return

    new_student = Student(student_id, fullname, age, email, department)
    students.append(new_student)

    with open("students.txt", "a") as file:
        file.write(f"{fullname},{email},{department}\n")

    print("Student registered successfully")


register_student()

# SQL Examples
# INSERT INTO students(fullname, age, email) VALUES ('Larry', 18, 'larry@gmail.com');
# SELECT * FROM students;
```

2. Hospital Patient Record System

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age


class Patient(Person):
    def __init__(self, patient_id, name, age, gender):
        super().__init__(name, age)
        self.patient_id = patient_id
        self.gender = gender


class Doctor(Person):
    def __init__(self, doctor_id, name, age, specialty):
        super().__init__(name, age)
        self.doctor_id = doctor_id
        self.specialty = specialty


appointments = []

def book_appointment(patient, doctor, date):
    for appointment in appointments:
        if appointment["doctor"] == doctor and appointment["date"] == date:
            print("Doctor already booked for that date")
            return

    appointments.append({
        "patient": patient,
        "doctor": doctor,
        "date": date
    })

    print("Appointment booked successfully")


patient1 = Patient("P001", "John", 22, "Male")
doctor1 = Doctor("D001", "Dr Smith", 45, "Cardiology")

book_appointment(patient1.name, doctor1.name, "2026-05-12")

# SQL Examples
# SELECT * FROM patients;
# UPDATE patients SET age = 30 WHERE patient_id = 'P001';
```

3. E-Commerce Inventory & Order System

```
class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock


class Customer:
    def __init__(self, username, password):
        self.username = username
        self.password = password
        self.cart = []

    def add_to_cart(self, product, quantity):
        if quantity > product.stock:
            print("Insufficient stock")
            return

        self.cart.append((product, quantity))
        print("Added to cart")
    def checkout(self):
```

```
        total = 0

        for item in self.cart:
            product, quantity = item
            total += product.price * quantity
            product.stock -= quantity

        print(f"Total Amount: {total}")


laptop = Product(1, "Laptop", 500000, 10)

customer = Customer("larry", "python123")

customer.add_to_cart(laptop, 2)
customer.checkout()

# SQL Examples
# INSERT INTO products(name, price, stock) VALUES ('Laptop', 500000, 10);
# UPDATE products SET stock = stock - 2 WHERE id = 1;
```

4. Banking/ATM System

```
class Account:
    def __init__(self, account_number, fullname, pin, balance=0):
        self.account_number = account_number
        self.fullname = fullname
        self.__pin = pin
        self.balance = balance
        self.transactions = []

    def verify_pin(self, pin):
        return self.__pin == pin

    def deposit(self, amount):
        self.balance += amount
        self.transactions.append(f"Deposited {amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance")
            return

        self.balance -= amount
        self.transactions.append(f"Withdrew {amount}")

    def transfer(self, receiver, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return

        self.balance -= amount
        receiver.balance += amount

        self.transactions.append(f"Transferred {amount}")
        receiver.transactions.append(f"Received {amount}")


acc1 = Account("1001", "Larry", "1234", 5000)
acc2 = Account("1002", "John", "2222", 2000)

acc1.transfer(acc2, 1000)

print(acc1.balance)
print(acc2.balance)

# SQL Examples
# UPDATE accounts SET balance = balance - 1000 WHERE account_number = '1001';
# UPDATE accounts SET balance = balance + 1000 WHERE account_number = '1002';
```

5. Student Result Processing Portal

```
class Course:
    def __init__(self, code, unit):
        self.code = code
        self.unit = unit


class Result:
    def __init__(self, course, score):
        self.course = course
        self.score = score

    def grade_point(self):
        if self.score >= 70:
            return 5
        elif self.score >= 60:
            return 4
        elif self.score >= 50:
            return 3
        elif self.score >= 45:
            return 2
        else:
            return 0


class Student:
    def __init__(self, name):
        self.name = name
        self.results = []

    def add_result(self, result):
        self.results.append(result)

    def calculate_gpa(self):
        total_points = 0
        total_units = 0

        for result in self.results:
            total_points += result.grade_point() * result.course.unit
            total_units += result.course.unit

        gpa = total_points / total_units
        return round(gpa, 2)


python_course = Course("CSC101", 3)

student = Student("Larry")
student.add_result(Result(python_course, 75))

print("GPA:", student.calculate_gpa())

# SQL Examples
# SELECT * FROM results WHERE student_id = 1;
# UPDATE results SET score = 80 WHERE id = 3;
```

These implementations are intentionally simplified. In real Django applications, the same logic would be handled using models, views, forms, templates, authentication systems, ORM queries, middleware, sessions, and database transactions.